Assignment Questions

Voltkart · Session 1 — Advanced SQL for Modern Data Engineering

Before you start

> **Note** Engine is **SQL Server (T-SQL)**, run in SSMS. Use `order_status = 'Completed'` for revenue and performance questions unless a question says otherwise. The questions build up — warm-up first, then core, then stretch. Estimated effort is 2–3 evenings.

Warm-up

1. Voltkart's commercial team wants a quick look at the biggest sales. Return the **top 20 completed orders by value**, with the customer and the sales rep behind each one. **Required output:** `order_id`, `order_date`, `customer_name`, `sales_rep_name`, `order_total`.

2. Marketing wants to re-engage people who registered but never bought. List the customers who have **never placed an order**. **Required output:** `customer_id`, `customer_name`, `signup_date`.

3. Merchandising wants the stars of each category. For **every category**, return the **top 3 products by completed revenue**. **Required output:** `category_name`, `product_name`, `total_revenue`, `revenue_rank`.

Core

4. Finance wants the revenue trend with momentum. Produce **monthly completed revenue** with a **cumulative running total** and the **month-over-month % change**. **Required output:** `order_month` (YYYY-MM), `monthly_revenue`, `running_total`, `mom_pct_change`.

5. The CRM team wants to size up the customer base by value. Split customers into **four quartiles by lifetime completed spend**, and for each quartile report how many customers fall in it and their average spend. **Required output:** `spend_quartile`, `customer_count`, `avg_lifetime_spend`.

6. The catalogue team needs to see everything under a part of the tree. Using a **recursive query**, list **all categories in the subtree rooted at 'Computers'** at any depth, with how deep each sits and a readable path from Computers. **Required output:** `category_id`, `category_name`, `depth_level`, `category_path`.

7. Sales leadership wants performance rolled up the org chart. For **every employee**,

compute the **total completed order value generated by their whole team** — that is, themselves plus everyone reporting under them at any depth. A Sales Rep's team is just themselves; a manager's is their entire subtree. **Required output:** `employee_id`, `employee_name`, `role`, `team_total_revenue`.

8. The platform team is moving off nightly full reloads. You've been handed today's batch in `stg_orders_incr`. Write a **single MERGE** that performs an **incremental load** into `fact_orders`: update orders that changed, insert ones that are new. **Acceptance criteria:** one MERGE statement; matched orders are updated (status and total) and new orders inserted; include a verification query showing the `fact_orders` row count before versus after and a sample of updated rows.

Stretch

9. The catalogue source now emits a **change feed** (`cdc_product_changes`) with operation codes. Apply it to `dim_product` in a **single MERGE** that honours the codes: **I** inserts a new product, **U** updates an existing one, **D** deletes it. **Acceptance criteria:** one MERGE handling all three operations; include a verification query showing inserted, updated, and deleted products.

10. A report is slow. Here is the query analysts are running:

```
SELECT o.customer_id, COUNT(*) AS orders_2024,
       (SELECT SUM(oi.line_amount)
          FROM fact_order_items oi
          JOIN fact_orders o2 ON o2.order_id = oi.order_id
         WHERE o2.customer_id = o.customer_id) AS lifetime_value
FROM fact_orders o
WHERE YEAR(o.order_date) = 2024
GROUP BY o.customer_id;
```

Turn on `SET STATISTICS IO, TIME ON` and "Include Actual Execution Plan", run it, then **make it fast**: explain what the plan is doing, rewrite the query, and add an index that helps. **Required output:** the rewritten query, the `CREATE INDEX` statement(s), and a short note giving the **logical reads before versus after** and why the plan changed.

11. **(Bonus)** The retention team wants loyalty streaks. For each customer, find the **longest run of consecutive calendar months** in which they placed at least one completed order. **Required output:** `customer_id`, `customer_name`, `longest_streak_months`.